EYE DISEASE CLASSIFICATION

Detailed Technical Report

InceptionV3 Transfer Learning + Custom Classification Head

Project	Eye Diseases Classification
Classes	Cataract • Diabetic Retinopathy • Glaucoma • Normal
Dataset	Kaggle — gunavenkatdoddi/eye-diseases-classification
Framework	TensorFlow 2.19.0 / Keras
Input	256 × 256 × 3 RGB
Model	Pre-trained InceptionV3 + custom CNN/Dense head
Evaluation	Accuracy, Precision, Recall, AUC, F1-score

Source: Uploaded Jupyter/Colab notebook *Copy_of_Eye_Diseases_Classification_InceptionV3_95 (1).ipynb*.

Important: This report describes the recorded notebook run. It is a technical ML report, not a clinical validation or medical diagnostic certificate.

Table of Contents

1. Project Overview
2. Clinical Class Definitions
3. Dataset and Data Loading
4. Exploratory Data Analysis
5. Preprocessing and Data Augmentation
6. InceptionV3 Transfer Learning
7. Custom Classification Architecture
8. Class Weights and Label Encoding
9. Compilation and Training Strategy
10. Training Results and Learning Curves
11. Model Evaluation
12. Confusion Matrix Analysis
13. Classification Report
14. Strengths and Limitations
15. Reproducibility Notes
16. Conclusion
17. Source Traceability

1. Project Overview

The notebook implements a four-class eye-image classification pipeline. It uses a pre-trained InceptionV3 network as the feature-extraction backbone, truncates the backbone at the **mixed8** layer, and adds custom convolutional, normalization, pooling, dense and dropout layers. The final layer uses a four-unit softmax classifier.

Aspect	Notebook implementation
Problem type	Multi-class image classification
Classes	4
Dataset images	4,217
Training images	3,374 (80%)
Validation images	843 (20%)
Original sample size checked	512 × 512
Model input	256 × 256 × 3
Backbone	InceptionV3, ImageNet weights, include_top=False
Backbone cut point	mixed8
Final activation	Softmax

The notebook is designed around transfer learning: the general visual features learned from ImageNet are reused, while later layers are customized to the eye-disease classification task.

2. Clinical Class Definitions

Class	Description recorded in notebook
Cataract	Clouding of the eye lens that can reduce vision; the notebook notes aging and diabetes as risk factors and surgery
Diabetic Retinopathy	Diabetes-related damage to retinal blood vessels that may progress to vision loss.
Glaucoma	Group of diseases that damage the optic nerve and can cause permanent vision loss if not diagnosed early.
Normal	Healthy-eye category representing images without the pathological condition targeted by the other classes.

These definitions are reproduced at a high level from the notebook's introductory section; they are contextual descriptions rather than a clinical reference.

3. Dataset and Data Loading

The notebook downloads the Kaggle dataset using KaggleHub and reads the directory structure under **/kaggle/input/eye-diseases-classification/dataset**. The class names are discovered from directory names and sorted alphabetically.

Class	Training	Validation	Total
Cataract	836	204	1,040
Diabetic Retinopathy	884	217	1,101
Glaucoma	802	201	1,003
Normal	852	221	1,073
Total	3,374	843	4,217

Dataset creation uses **image_dataset_from_directory** with a 20% validation split, batch size 32, image size 256×256 and shuffling for the training subset. The notebook records 4,217 files across four classes.

A sample image is reported as 512×512 before the dataset loader resizes it to 256×256. The resulting batch shape is (32, 256, 256, 3).

Class balance: the classes are reasonably close in size, but not perfectly equal. The notebook therefore computes balanced class weights before training.

4. Exploratory Data Analysis

The notebook performs several basic EDA steps: class-label counting, pie-chart visualization, checking image shapes, displaying a 4×4 grid of training samples, and randomly visualizing five images from each class directory.

EDA step	Purpose
Label distribution	Identify class imbalance and confirm directory labels.
Batch/image shape	Verify dimensions expected by the neural network.
Sample grid	Visually inspect examples and label assignments.
Per-class visualization	Inspect representative images for Cataract, DR, Glaucoma and Normal.
Caching/prefetching	Improve input-pipeline performance during training.

The input pipeline is optimized using **cache()**, **shuffle(1000)** for training, and **prefetch(AUTOTUNE)**.

5. Preprocessing and Data Augmentation

The notebook defines a Keras Sequential augmentation pipeline. These transformations are applied before the InceptionV3 preprocessing step.

Transformation	Notebook setting	Purpose
RandomFlip	horizontal	Increase orientation variability.
RandomRotation	0.1	Small rotational variation (~±10%).
RandomZoom	0.1	Simulate mild scale variation.
RandomContrast	0.1	Vary contrast/lighting.
RandomBrightness	0.1	Simulate brightness differences.
RandomTranslation	0.1, 0.1	Small horizontal/vertical shifts.
GaussianNoise	0.05	Simulate sensor/noise variation.

After augmentation, the images are passed through **tensorflow.keras.applications.inception_v3.preprocess_input**, matching the preprocessing expected by the pre-trained InceptionV3 backbone.

6. InceptionV3 Transfer Learning

The backbone is created as **InceptionV3(include_top=False, weights='imagenet', input_shape=(256,256,3))**. The original ImageNet classification head is removed. The notebook then selects the internal **mixed8** layer and creates a truncated Functional model whose output is that layer's feature map.

The recorded model inspection shows the truncated Inception component producing a feature tensor of approximately **6 × 6 × 1280**. This feature representation is then passed to the custom classification head.

Fine-tuning: the notebook sets **truncated_model.trainable = True**, meaning the selected InceptionV3 portion is trainable during the task-specific training phase.

7. Custom Classification Architecture

The custom head is constructed with the Keras Functional API. The sequence is:

Stage	Layer / operation	Output / role
1	Input	256×256×3 RGB image
2	Data augmentation	Training-time variability
3	InceptionV3 preprocess_input	Backbone-compatible values
4	Truncated InceptionV3	6×6×1280 feature map
5	Conv2D	256 filters, 3×3, ReLU
6	BatchNormalization	Training stabilization
7	Conv2D	128 filters, 3×3, ReLU
8	BatchNormalization	Training stabilization
9	GlobalAveragePooling2D	128-dimensional vector
10	Dense	256, ReLU
11	Dropout	0.50
12	Dense	128, ReLU
13	Dropout	0.30
14	Dense	4, Softmax

Parameter summary	Recorded value
Total parameters	13,987,236
Trainable parameters	13,965,092
Non-trainable parameters	22,144
Approx. model size	53.36 MB

The custom layers are intended to adapt the general InceptionV3 representation to disease-specific patterns, while batch normalization and dropout provide regularization/stability.

8. Class Weights and Label Encoding

Balanced class weights are calculated from the training labels using scikit-learn's **compute_class_weight('balanced', ...)**. The resulting weights are passed to model.fit().

Class	Weight
Cataract (0)	1.008971
Diabetic Retinopathy (1)	0.954186
Glaucoma (2)	1.051746
Normal (3)	0.990023

Because the training labels are initially integer encoded, the notebook maps both training and validation labels to one-hot vectors with depth 4. This matches the four-unit softmax output and the categorical cross-entropy loss.

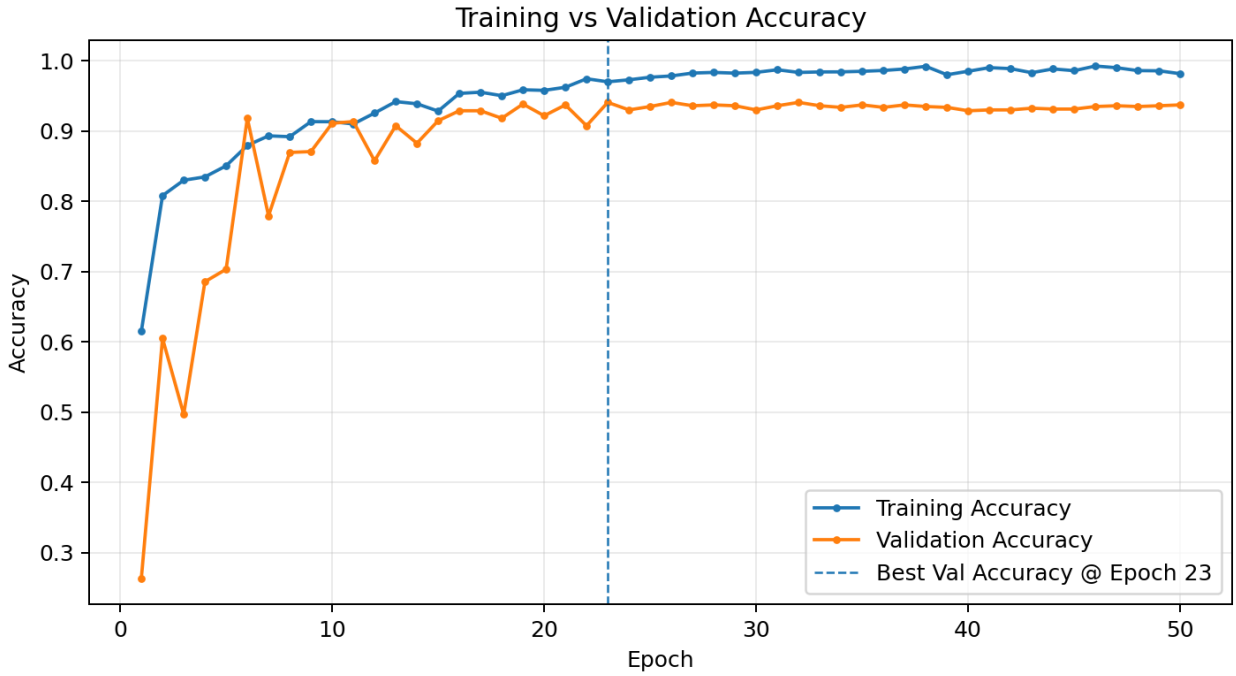
9. Compilation and Training Strategy

Component	Notebook setting
Optimizer	Adam
Initial compile learning rate	1×10■■
Loss	categorical_crossentropy
Metrics	Accuracy, Precision, Recall, AUC, macro F1-score
Maximum epochs	100
Model checkpoint	best_model.keras, monitored by val_accuracy
Early stopping	val_accuracy, patience=50, restore_best_weights=True, baseline=0.99, min_delta=0.001
Custom LR scheduler	0.001 × 0.5^(epoch//5)
Class weighting	Enabled

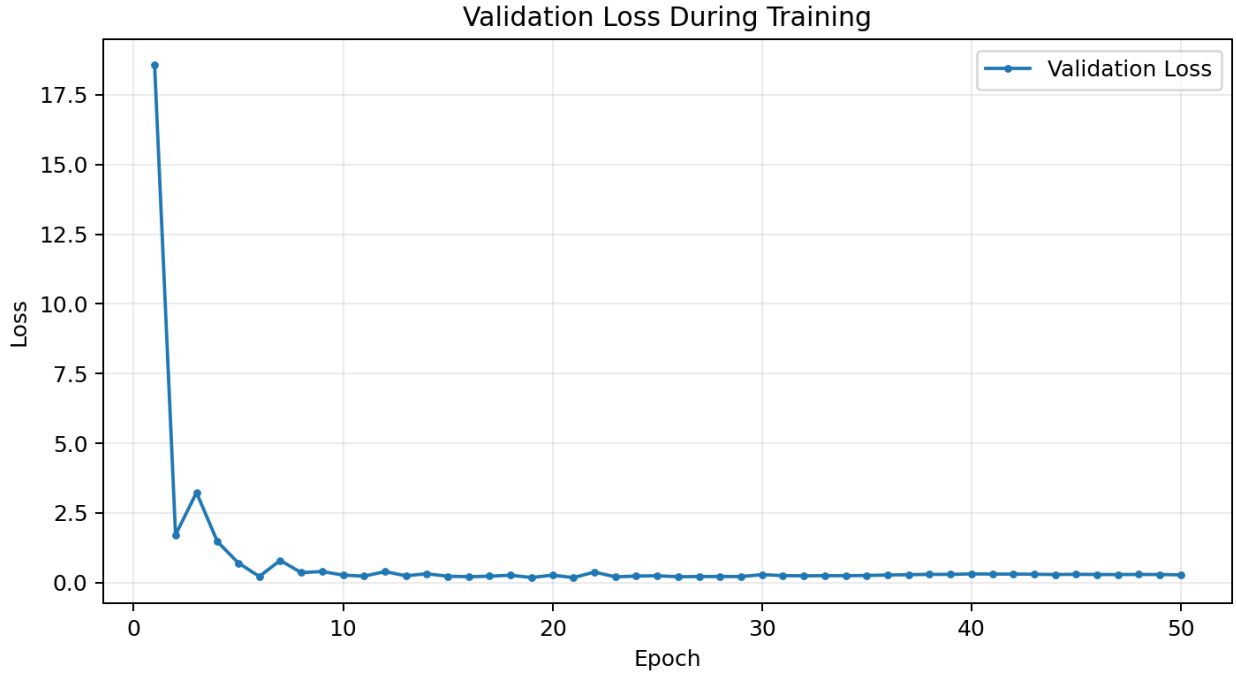
The training call explicitly supplies the class-weight dictionary and the callbacks for early stopping, checkpointing and the custom learning-rate schedule. The notebook also defines a ReduceLROnPlateau callback, but the recorded model.fit() call uses only EarlyStopping, ModelCheckpoint and LearningRateScheduler.

The first recorded epoch starts at training accuracy ≈61.49% and validation accuracy ≈26.33%. Performance rises substantially over subsequent epochs.

10. Training Results and Learning Curves



The highest recorded validation accuracy is **94.07% at epoch 23**. At that epoch, training accuracy is 97.00%, validation AUC is 0.9909, validation F1 is 0.9418, and validation loss is 0.2039.



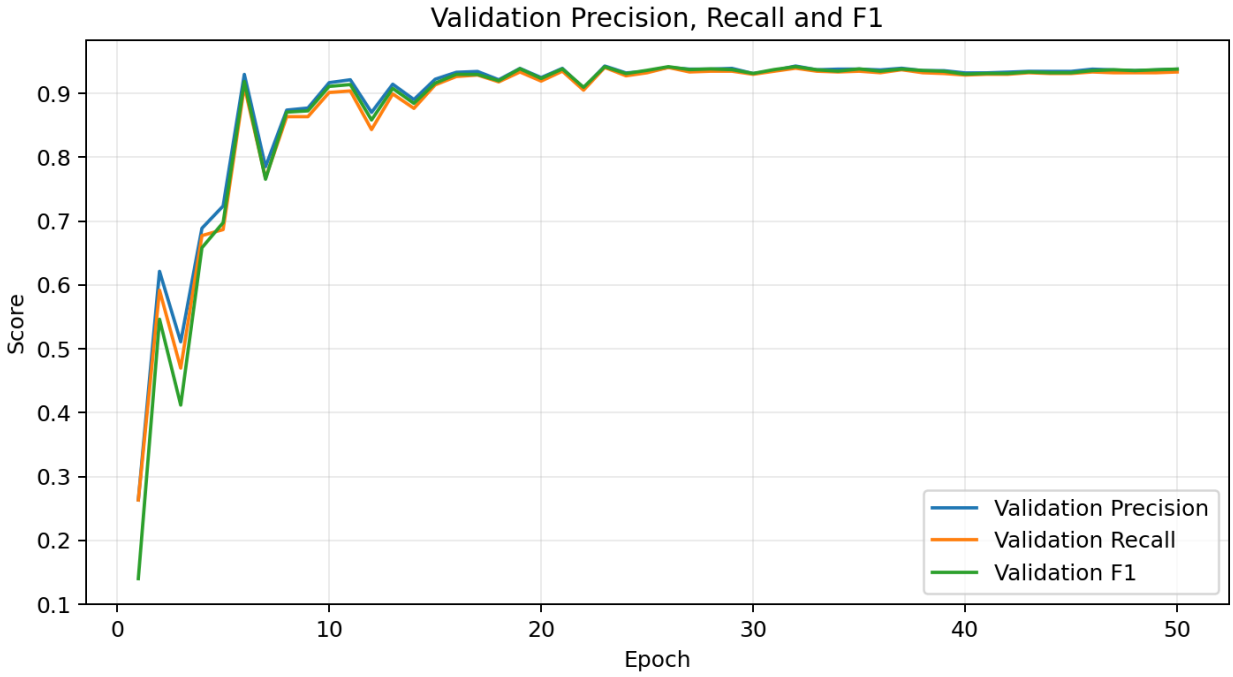
At the last recorded epoch (50), training accuracy is 98.15% while validation accuracy is 93.71%. The gap indicates some training/validation divergence in the later part of training, although the best checkpoint was retained.

11. Model Evaluation

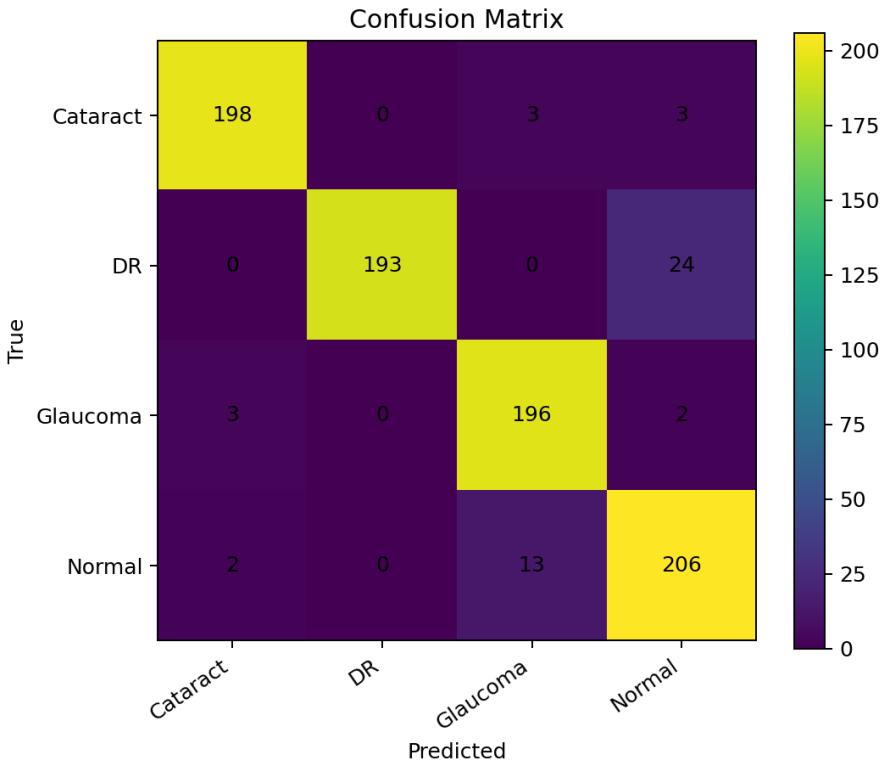
The notebook reloads **best_model.keras** and evaluates it on the validation dataset. These are the principal recorded evaluation results:

Metric	Value
Accuracy	0.9407 (94.07%)
AUC	0.9909
Macro F1-score	0.9418
Precision	0.9429
Recall	0.9407
Loss	0.2039
Validation samples	843

The evaluation output is consistent with the best validation checkpoint reported in the training history: epoch 23 reached 0.9407 validation accuracy and 0.9418 validation F1.



12. Confusion Matrix Analysis



The notebook records the following confusion matrix, with rows representing true classes and columns representing predicted classes in the class order used for evaluation: Cataract, Diabetic Retinopathy, Glaucoma and Normal.

True \ Pred.	Cataract	DR	Glaucoma	Normal
Cataract	198	0	3	3
DR	0	193	0	24
Glaucoma	3	0	196	2
Normal	2	0	13	206

The largest off-diagonal error count is **24 diabetic-retinopathy images predicted as normal**. The next notable confusion is **13 normal images predicted as glaucoma**. Cataract and glaucoma show relatively few cross-class errors in this validation set.

13. Classification Report

Class	Precision	Recall	F1	Support
Cataract	0.98	0.97	0.97	204
Diabetic Retinopathy	1.00	0.89	0.94	217
Glaucoma	0.92	0.98	0.95	201
Normal	0.88	0.93	0.90	221
Accuracy			0.94	843
Macro Avg.	0.94	0.94	0.94	843
Weighted Avg.	0.94	0.94	0.94	843

Interpretation: Cataract has strong precision and recall (0.98/0.97). Diabetic Retinopathy has perfect recorded precision (1.00) but lower recall (0.89), consistent with the confusion matrix showing 24 DR images classified as normal. Glaucoma has high recall (0.98), while Normal has the lowest precision among the four classes (0.88).

These metrics are validation-set results from the notebook and should not be interpreted as evidence of clinical diagnostic performance on unseen hospitals, devices, populations or real-world workflows.

14. Strengths and Limitations

Strengths

Strength	Why it matters
Transfer learning	Reuses a strong visual representation rather than learning all features from scratch.
Custom head	Adapts the backbone to the four target classes.
Data augmentation	Improves variability and can reduce overfitting.
Multiple metrics	Accuracy alone is supplemented with precision, recall, AUC and F1.
Checkpointing	Preserves the best validation-accuracy model.
Class weighting	Addresses moderate class imbalance explicitly.
Confusion matrix	Shows which classes are being confused, not only the aggregate score.

Limitations / risks

Issue	Implication
Validation-only evaluation	The notebook evaluates the held-out validation split; an independent external test set is not recorded.
Potential distribution shift	Performance may change with different cameras, imaging protocols, demographics or clinical settings.
Class ambiguity	Some eye conditions can share visual characteristics; errors require clinical interpretation.
Training/validation gap	Later epochs show training accuracy approaching 99% while validation accuracy remains around 93–94%.
Medical use	The model should be treated as a research/decision-support component, not a standalone diagnosis.

15. Reproducibility Notes

The notebook records a Python 3.11.11 environment, TensorFlow 2.19.0, Kaggle/Colab GPU execution, and a T4 GPU designation in metadata. The dataset is imported through KaggleHub.

Important implementation details to reproduce the recorded pipeline include: 256×256 input resizing; seed 123 for the training split and seed 42 for the validation loader; batch size 32; the listed augmentation sequence; ImageNet InceptionV3 weights; the mixed8 truncation point; one-hot labels; Adam; categorical cross-entropy; class weights; and the callback configuration.

Notebook consistency note: a later cell in the notebook creates a manually defined dictionary named *ideal_training_data* containing a different summary (25 epochs, best validation accuracy 0.9612, best model at epoch 22). Those values do not match the actual recorded training/evaluation outputs above, which show 50 recorded epochs and a best validation accuracy of 0.9407 at epoch 23. This report deliberately uses the recorded model outputs for performance claims and treats the ideal summary as non-authoritative.

16. Conclusion

The uploaded notebook demonstrates a complete deep-learning workflow for four-class eye-image classification. It combines dataset inspection, augmentation, InceptionV3 transfer learning, a custom CNN classification head, class weighting, multi-metric monitoring, checkpointing and detailed post-training evaluation.

The recorded best validation checkpoint achieves **94.07% accuracy**, **0.9418 macro F1** and **0.9909 AUC** on 843 validation images. The main error pattern is reduced recall for diabetic retinopathy because a portion of DR images are predicted as normal.

For a stronger research result, the next logical steps would be independent test-set evaluation, external validation, patient-level split verification, subgroup analysis, calibration analysis, explainability (for example, Grad-CAM), and clinical review before any real-world diagnostic deployment.

17. Source Traceability

All project-specific facts in this report are grounded in the uploaded notebook. Key source areas include the dataset setup and class counts, augmentation pipeline, InceptionV3 architecture, model summary, class weights, training configuration, recorded epoch outputs, validation evaluation, confusion matrix and classification report.

Notebook area	Information used
Cells 1–8	Project scope, disease classes, libraries, TensorFlow version and dataset class names.
Cells 9–15	Image dimensions, train/validation split, counts and sample shapes.
Cells 21–22	Input pipeline optimization and augmentation.
Cells 23–30	InceptionV3 backbone, mixed8 truncation, custom head and parameter counts.
Cells 31–34	Class weights, one-hot labels, compilation and callbacks.
Cell 35	Recorded training history.
Cells 36–39	Training/validation metric summaries and plots.
Cells 40–47	Best-model evaluation, confusion matrix and classification report.
Cell 54	Manually defined ideal-training summary; reported separately because it conflicts with actual recorded results.

Generated as a detailed technical study/report from the uploaded notebook. No external clinical claims were added to the recorded results.